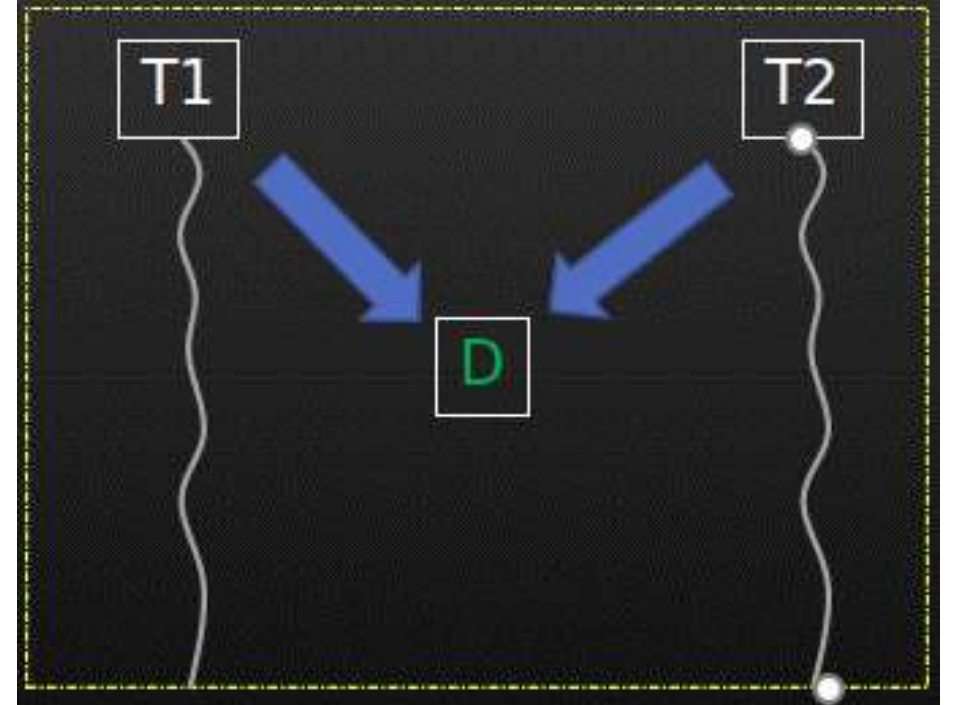


EADL- Thread Synchronization

<https://community.ruggedboard.com>

Thread Synchronization

- The hardest and most important aspect of Multi-threading is – Thread Synchronization
- Thread Synchronization is required in multi-threaded programs whenever multi-threads compete to
 - perform conflicting operations (Read-Write Or Write-Write) on a shared resource
 - Shared Resources :
 - Heap Data Structures
 - Global Variable
 - File Descriptors (opened Files, Sockets)
 - Receiving Data from External Sources via multiple inlets
- Let us first try to understand the problems we would have if we don't have Thread-Synchronization



Thread Synchronization

- Consider a multi-threaded process P having two threads T1 and T2
- Also, consider a Process P maintains a linked list of integers
- Thread T1 and T2 are scheduled on the CPU(s) in any order – do not assume determinism/pattern !

Thread T1	Thread T2
<pre>node_t * get_node_from_list(int a) { node_t *node = search_node(list, a); return node; }</pre>	<pre>void delete_node_from_list(int a) { node_t *node = search_node(list, a); remove_node(node); free(node); }</pre>

- Concurrent access of shared data structures between Multiple-Threads opens a window during which data is in-consistent – Root Cause is Concurrency
- The region in code where shared data is accessed by multiple threads are called Critical Sections
- Goal : Concurrency + Data Consistency

Critical Section

```
foo() {  
...  
...  
global_var1++;  
global_var2++;  
...  
...  
}
```

- Code Excerpt accessing the shared Data are critical sections
- Shared Data –
 - Global Variables
 - Heap Data Structures
 - Static Variables
- Rule of Thumb : Critical Sections Must be executed by Concurrent threads but by one and only one thread at a time
 - Unexpected behavior
 - Segment Fault
 - Data Corruption
 - Any abnormal behavior
- Thread Synchronization – Identify CS and apply several techniques to prevent unprotected Concurrent access to shared resources by several threads of the process

```
foo() {  
    static int i = 1;  
    I1 : i = i + 1 ;  
    I2 : i = i + 2 ;  
    I3 : print i ;  
}
```

Critical Section

- If the function `foo( )` is executed by three concurrent threads, what are possible outputs of the program ?
- Different outputs based on sequence of threads execution
- If `i` is changed to local variable, there is no Critical Section
- More Examples of CS :
 - Two or more threads trying to update same database
 - Two or more threads trying to read/write into same memory buffers
- CS are critical in the sense, if they are allowed to be accessed by multiple threads without deploying thread synchronization techniques, the application sooner, or later, is bound to fail

Thread Synchronization -> Mutexes

- Mutexes are Thread Synchronization constructs/tools which provide Mutual Exclusivity while accessing a critical Section by multiple concurrent threads

Analogy :

- Mutexes are like Keys to the locker, whoever wants to access the locker need to have keys
- Whoever do not have keys, cannot access locker and has to wait
- Whoever is accessing the locker, need to handover the keys when done accessing the locker
 - Locker = Critical Section
 - Whoever = Threads
 - Keys = Mutexes
- Half of the work is done, when you are able to map Thread Synchronization Concepts to real world scenarios

```
pthread_mutex_t mutex;  
  
foo() {  
    ...  
    ...  
    ...  
l1 pthread_mutex_lock(&mutex);  
l2 /* Critical Section */  
l3 pthread_mutex_unlock(&mutex);  
    ...  
    ...  
}
```

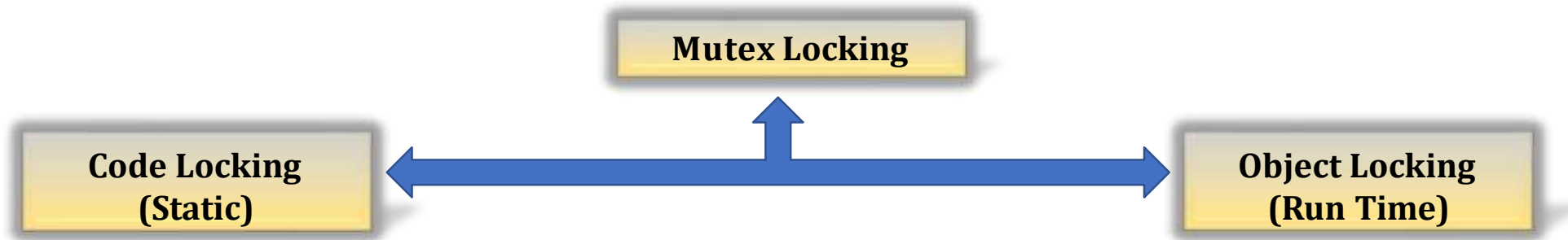
Grant Access to C.S to only one and one thread at a time

- Whichever threads locks the Mutex, shall be able to enter into CS
- If a thread tries to lock already locked mutex, thread is blocked
- Multi-threads may be blocked by same Mutex
- When thread exits out the Critical Section, it MUST unlock the Mutex
- Among many threads waiting for the same Mutex, Mutex shall be granted to only one depending on several factors such as :(Thread Priority, OS scheduling policy)
- Thread must not intentionally die while holding the Mutex lock, that Mutex become unusable for forever
- Use of Mutexes causes Threads to block and unblock. More the blocking and unblocking of threads, More Scheduling work overhead OS, poorer shall be the application performance
- Bigger the size of CS, larger the time the blocked threads would have to wait, hence again poorer shall be application performance.

Thread Synchronization -> Mutex locking Rules

- If T1 locks a mutex M, only T1 can unlock it
- T1 cannot unlock an already unlocked mutex -> Undefined behavior
- If T1 locks the Mutex M, T2, T3 ... will be blocked if they tries to lock M
- If T1, T2 ... are blocked to acquire lock on already locked Mutex M, the OS scheduling policy will decide which thread (i.e. among T1 or T2 ..) would acquire the lock on M when M is unlocked by its owner
- If thread T attempts to Double lock the Mutex M, it will self-deadlocked
- Mutexes Must be unlocked in LIFO Order (Recommendation)

Thread Synchronization



- If you need to protect a code snippet against concurrent thread unsafe access, go for Code locking
- If you need to protect Object against concurrent thread unsafe access, go for Object locking

```
static pthread_mutex_t mutex; foo() {  
    ...  
    ...  
    ...  
    pthread_mutex_lock(&mutex);  
  
    /* Critical Section */  
    pthread_mutex_unlock(&mutex);  
}
```

Code locking

Mutex are defined at Src file level

```
static pthread_mutex_t mutex;
```

```
void pkt_receive (char *pkt, int pkt_size)  
{  
    pthread_mutex_lock(&mutex);  
    memset (global_buffer, 0, sizeof(global_buffer));  
    memcpy (global_buffer, pkt, pkt_size);  
    forward_pkt (global_buffer, pkt_size);  
    pthread_mutex_unlock(&mutex);  
}
```

Example of Code locking

Data Locking (Run Time Locking)

```
foo() {  
...  
...  
...  
pthread_mutex_lock(&list->mutex);  
/* Critical Section */  
/* perform_operation(&list); */  
pthread_mutex_unlock(&list->mutex);  
}
```

Data Structure locking
Every Data Structure is associated with its own Mutex (acts like a body-guard)

```
void  
delete_node_from_list ( list lst, int a ) {  
  
I5 pthread_mutex_lock(&lst->mutex);  
I6     node_t *node = search_node( lst, a);  
I7  
I8     remove_node(node);  
I9  
I10    free(node);  
I11 pthread_mutex_unlock(&lst->mutex);  
}
```

Note: student list ----> T1
Employee list ----> T2 } Not critical

student list ----> T1
Student list ----> T2 } critical

Mutex APIs

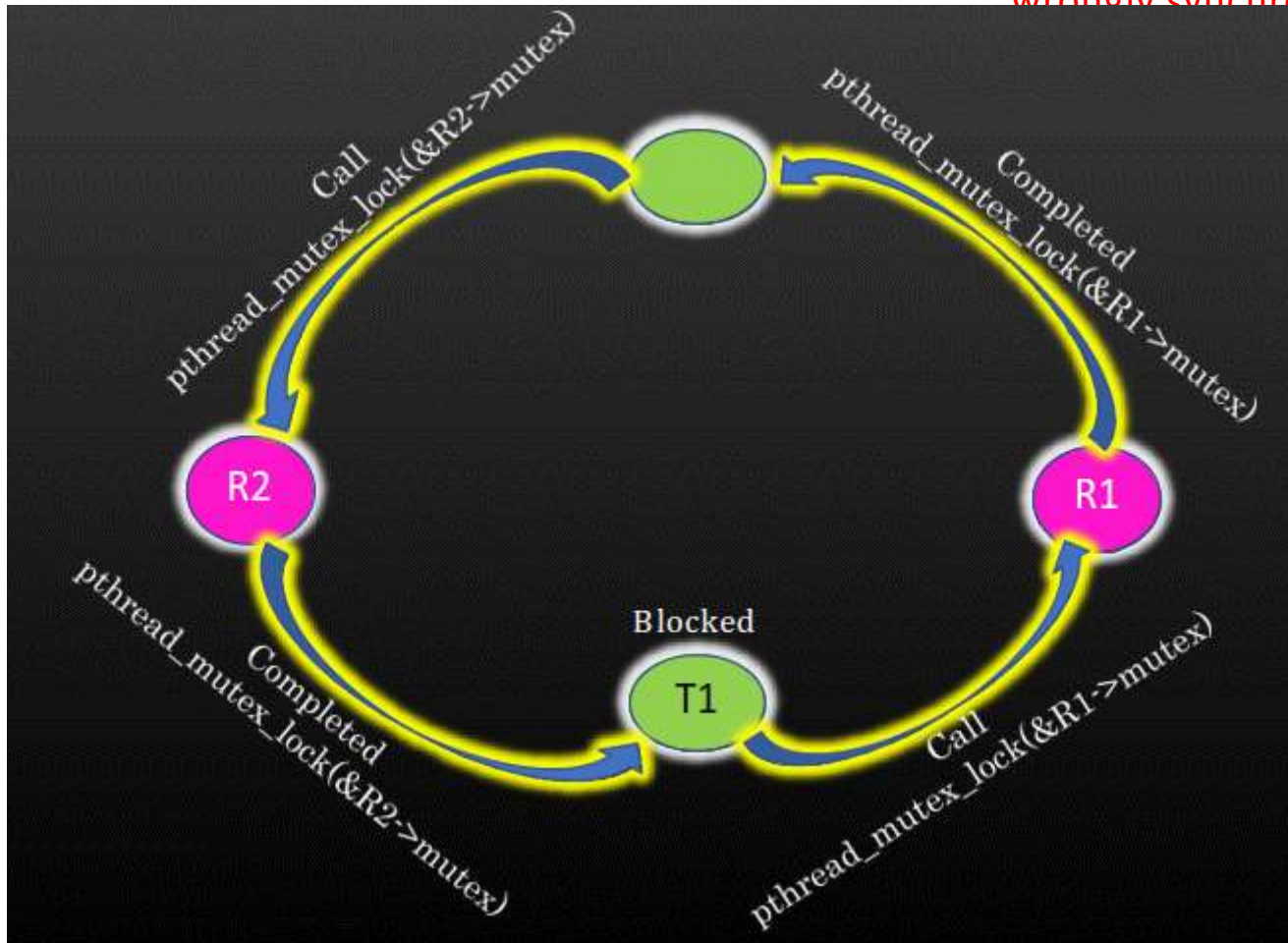
Declaration	: <code>pthread_mutex_t my_mutex;</code>
Initializing	: <code>pthread_mutex_init(&my_mutex, NULL);</code>
Destruction	: <code>pthread_mutex_destroy(&my_mutex);</code>
Locking and Unlocking the Mutex :	<code>pthread_mutex_lock(&mutex);</code> <code>pthread_mutex_unlock(&mutex);</code>

Note : Mutex must never be memcpy-ied --> Undefined behavior !

Programs

Thread Synchronization -> Deadlock

- Deadlock is a situation where nobody makes a progress, and gets blocked for forever
- Threads may undergo Deadlock if they are wrongly synchronized



Deadlocked !

Each Is waiting other to release the Resource

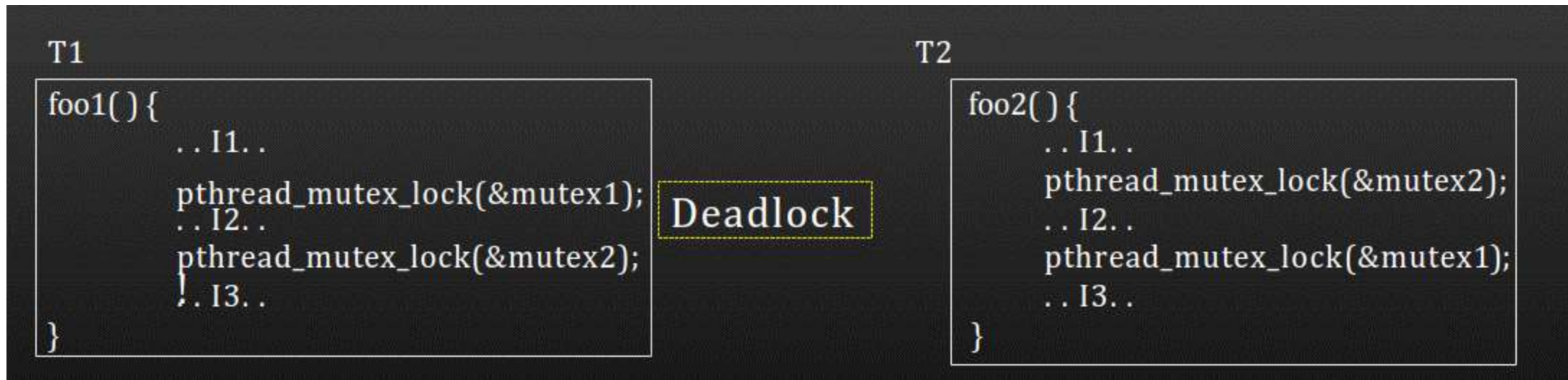
Four Necessary conditions for DeadLock

1. **Mutual Exclusion:** One or more than one resource are non-shareable (Only one thread can use at a time)
2. **Hold and Wait:** A thread is holding at least one resource and waiting for other resources.
3. **No Preemption:** A resource cannot be taken from a thread unless the thread releases the resource.
4. **Circular Wait:** A set of threads are waiting for each other in circular form.

Thread Synchronization

Thread Synchronization -> Deadlock -> Inconsistent Locking Order

- There can be scenarios where a thread needs to lock multiple mutexes
- The order in which a given thread locks multiple Mutex Matter



- The order of locking the mutexes must be preserved
- Order don't matter, but whatever order is there, same order should be followed through out the code
- It is recommended to unlock the Mutexes in LIFO order
- All four necessary conditions for deadlock to occur are true in this example as well.

Condition Variables

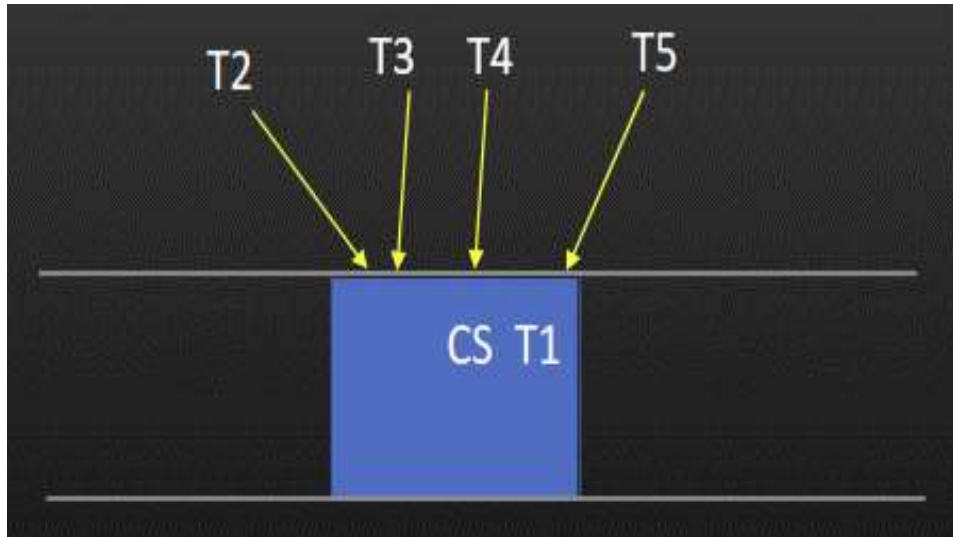
Thread Synchronization -> Condition Variables

Condition Variables allow us to have finer control over taking the decision on when and which competing thread to block/resume

Eg : Thread T1 finds Queue Empty, it wants to wait until Queue has some element in it
CV allows threads to get itself blocked or wake up when certain condition is met

Analogy:

- You would want to wait from having a delicious meal, unless it is cooked fully
- You would not buy expensive headphones until your next salary
- We always wait in our day to day life until some condition is satisfied, right ?
It's Natural.



When T1 leaves the CS, the lock is granted to the blocked thread chosen by the kernel's scheduling Policy.

Programmer has little control as to which thread should go next

- Mutexes only says –
Go ahead if you have access, wait if you don't
Like - *Yes Or No !*
- Using Mutexes we can't implement below logic :
If Queue is empty, wait until the Queue has some Element in it
- Mutex cannot implement condition-based blocking and wake up of threads,
For this CV is required
- CV + Mutex Combo is what it all takes to implement any advanced thread synchronization Scheme :
Monitors, Producer Consumer, Dining philosopher,
Thread Scheduler, Semaphores, Wait Queues, Barriers, etc

Thread Synchronization -> Condition Variables vs Mutex

- Mutex grant an access to the resource if it is not locked already,
- CVs allows threads to inspect the resource state and decide if it wants to wait for favorable resource state

Mutexes

- Access the laptop if it is not used by somebody else

Only Mutual Exclusion

```
pthread_mutex_lock(&laptop->mutex);  
  
    /* Do whatever you want */  
  
pthread_mutex_unlock(&laptop->mutex);
```

Condition Variables

- Access the laptop only if it not used by somebody else and if it has internet connection
- Mutual Exclusion + Custom Condition

```
pthread_mutex_lock(&laptop->mutex);  
  
    if (!laptop->internet_connection){  
        wait( cv, &laptop->mutex);  
    }  
    // use laptop  
pthread_mutex_unlock(&laptop->mutex);
```

Thread Synchronization -> Condition Variables

- CV are not used for Mutual Exclusion, they are for Co-ordination (Signaling)
- First Let us understand the basics of CVs, how they are used and then we dive deep into advanced application of CVs
- To put it simple :
 - Using CVs, a thread can block itself (`pthread_cond_wait`)
 - Using CVs, a thread can signal already blocked thread (blocked by CV) to resume (`pthread_cond_signal`)
- Let us understand the concept of Wait and Signals – The end result of Invention of CVs

Thread Synchronization -> Condition Variables -> Wait and Signal Steps

- Blocking a thread using CV
- A thread blocks itself using CV in 2 steps :
 - Step 1 : Lock a mutex
 - Step 2 : invoke `pthread_cond_wait`

T1

```
pthread_mutex_t  
mutex; pthread_cond_t cv;
```

```
pthread_mutex_init (&mutex,  
NULL); pthread_cond_init (&cv, NULL);
```

```
...  
printf ("T1 is getting  
blocked"); pthread_mutex_lock(&mutex);  
pthread_cond_wait(&cv, &mutex);  
printf ("T1 is  
awakened"); pthread_mutex_unlock(&mutex);
```

- Whoever invokes `pthread_cond_wait( )`, gets blocked
- Romeo and Juliet, CV and Mutex Variable

- When thread invokes `pthread_cond_wait( )`, two things happen :
 - Thread gets blocked (job of CV)
 - Mutex ownership is snatched from calling thread and is declared available
- When blocked Thread receives signal:
 - It slips into *ready to execute state* and wait for mutex release
 - It is given a lock on mutex as soon as mutex is released by signaling thread
 - Thread resumes execution

T2

```
...  
pthread_mutex_lock(&mutex);  
pthread_cond_signal(&cv); pthread_mutex_lock(&mutex);
```

Signaling a blocked thread using CV involves 3 steps :

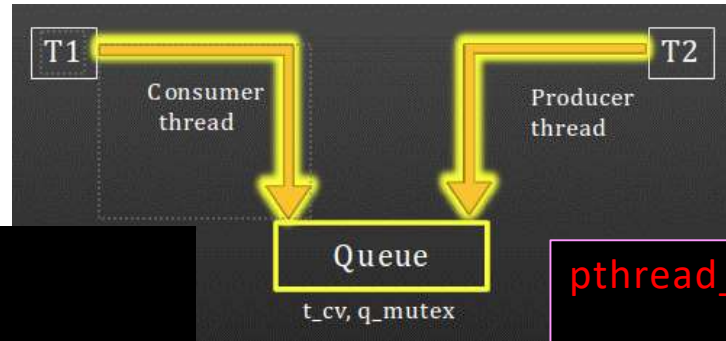
- Lock mutex
- Invoke `pthread_cond_signal`,
- unlock mutex

Thread Synchronization

Mutex + CV + Predicate

T1 (Consumer Threads)

```
S1 pthread_mutex_lock(&q_mutex);  
S2 if (q_empty(q)) {  
S3   pthread_cond_wait(&t_cv, &q_mutex);  
}  
S4 /* perform Ops on Queue (C.S) */  
S5 pthread_mutex_unlock(&q_mutex);
```



T2 (Producer Thread)

```
pthread_mutex_lock(&mutex); S6  
if ( !q_full (q) ) { S7  
    produce_and_enqueue(q);  
    pthread_cond_signal (&t_cv S8  
);  
}  
pthread_mutex_lock(&mutex); S9
```

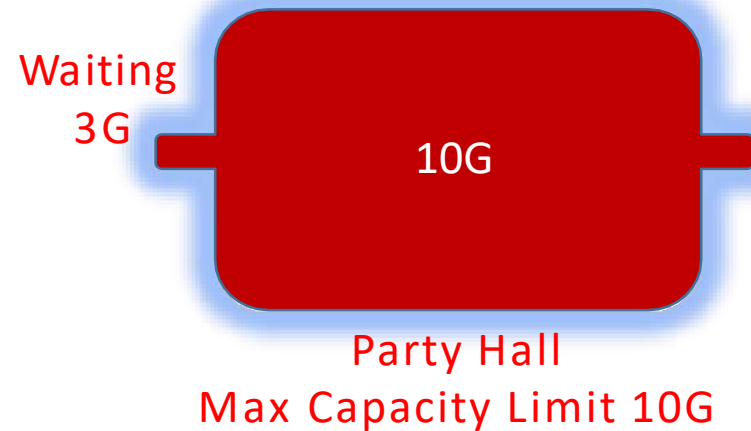
- To exclusively perform S2 (predicate), step S1 (Mutex locking) is required
- Predicate is a condition which tells the thread whether it has to wait or not
- If the condition for wait is true (S2), thread blocks using CV (S3)
- T2 check the resource state (S7) exclusively (S6), and produce the new elem
- T2 sends signal to T1 (S8), unlock mutex (S9)
- Step S4 is the C.S which T1 executes when woken up
- Step S5, explicitly unlock the shared data when done

Semaphores

Thread Synchronization

Semaphores Analogy

- You are throwing your Bday party and only 10 guests can accommodate in a party hall
- But 13 Guests arrived ! You were bad organizer indeed !



Permit # is 10

When one guest leaves, 1 Guest from Waiting list can enter into hall !

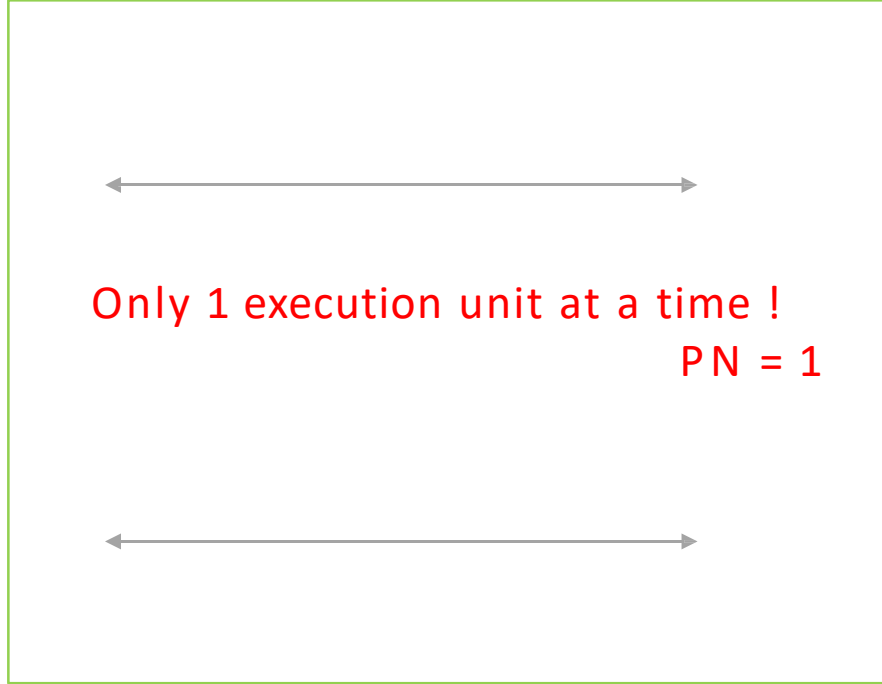
This is where Semaphores comes into picture

Guests - Execution Unit

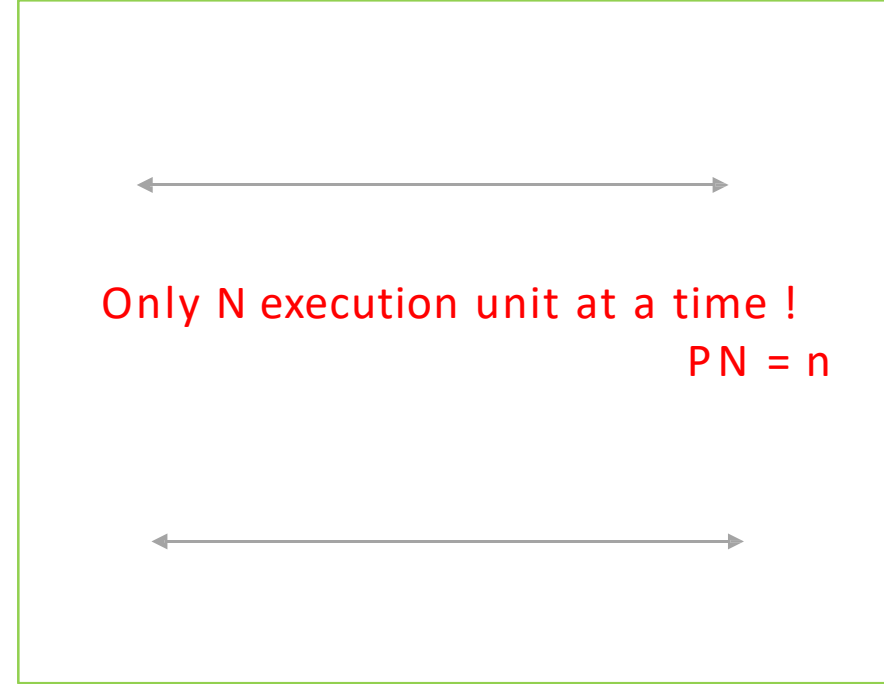
Party Hall - C.S, S

Semaphores Vs Mutex

Mutexes



Semaphores (n)



- If Semaphore S is initialized with $n = 1$, then Semaphore = Mutex in terms of functionality
- Therefore, Mutexes are also called binary semaphores
- Let us discuss next how semaphores work ...

Semaphores API's

```
#include <semaphore.h>
```

```
sem_t sem; /* Declare Semaphore Variable */
```

```
sem_init (sem_t *sem, int pshared, int permit counter);  
/* Initialize the Semaphore */
```

```
sem_wait (sem_t *sem);  
/* Block the calling thread if Semaphore is not available */
```

```
sem_post (sem_t *sem);  
/* Signal the blocked thread on semaphore */
```

```
sem_destroy (sem_t *sem);  
/* Destroy the semaphore after use */
```

Thread Synchronization

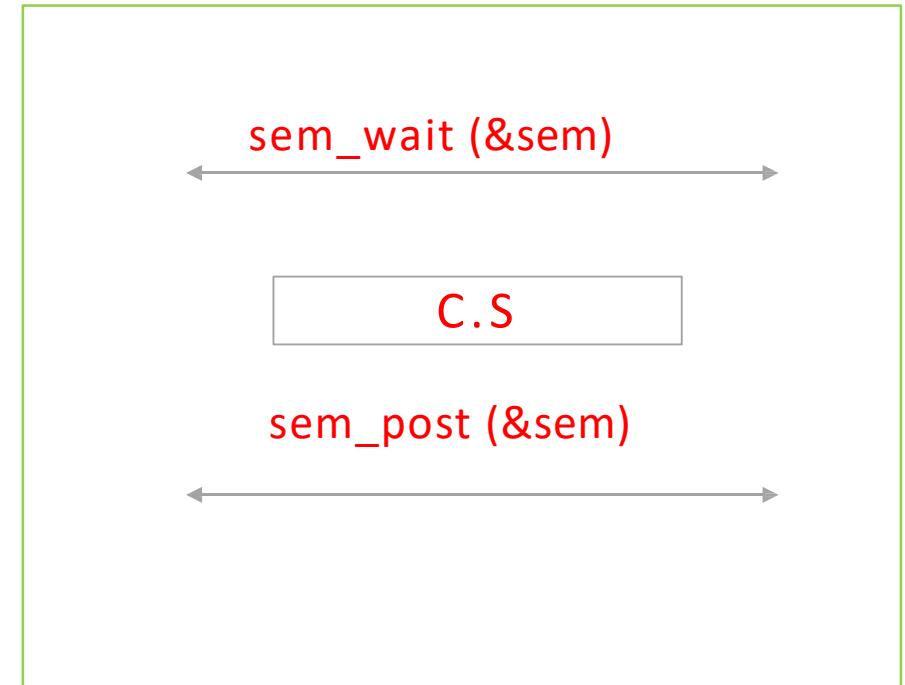
`sem_wait (sem_t *sem);`

- Unconditionally decrease the PC of the Semaphore
- If $PC < 0$ after decrement, block the calling thread

`sem_post (sem_t *sem);`

- Unconditionally Increase the PC of the semaphore
- Send signal to threads blocked on `sem_wait` , if any
- End Goal : Allow at-most 'n' threads to execute concurrently in C.S
- If $n = 1$, Binary semaphore which is same as mutex, but only difference is, semaphore can be unblocked (= `sem_post`) by a different thread

`sem_init(&sem, n )`



Semaphore Problem

- Create 5 threads
- But allow only max 2 threads to execute inside C.S concurrently at a time
- You can FAKE C.S code by printing some msg by the executing thread

Thank You



Developer
Wiki





Attribution 4.0 International (CC BY 4.0)

This is a human-readable summary of (and not a substitute for) the [license](#). [Disclaimer](#).

You are free to:

Share — copy and redistribute the material in any medium or format

Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

